Machine-Learning-Engineer-Assessment

Freight Rate Prediction | EDA, Feature Engineering, Modeling, Validation & Reproducibility

FINAL REVIEWED TECHNICAL REPORT

48,000	12,000	5.53%	171
labeled training rows	final validation rows	local validation MAPE	best boosting iteration

Verification scope

All numerical facts were re-checked against the uploaded raw CSVs, processed CSVs, executed notebooks, prediction outputs, validation template, December inputs, and scorer artifact. Figures labeled EXACT NOTEBOOK OUTPUT are the original embedded notebook images, not recreated charts.

01 Executive Summary

What was built, how it was evaluated locally, and what the supplied scorer verified.

This project predicts freight posted rates for future loads. The workflow begins with 48,000 labeled loads from January-October 2025 and produces predictions for 12,000 unlabeled loads from November-December 2025. Work was organized into EDA, cleaning, feature engineering, chronological local validation, LightGBM training, final retraining, prediction generation, a fixed December scenario, and scorer validation.

$129.79	$640.94	5.53%	5.42%
LightGBM MAE	LightGBM RMSE	LightGBM MAPE	LightGBM WAPE



Local performance

On the September-October time-based validation holdout, LightGBM reduced MAE by 88.7% relative to the median-rate baseline. The supplied scorer later validated all 12,000 final predictions and all 31 December rows. Spotter calculates the hidden final evaluation after submission; no hidden score is claimed here.

02 Problem Definition & Data

Prediction target, input fields, chronology, and required outputs.

Item	Training	Final validation
Rows	48,000	12,000
Raw columns	14	13
Date range	2025-01-01 to 2025-10-31	2025-11-01 to 2025-12-31
Target	posted_rate	Not provided
Load IDs	Unique	Prediction IDs match template exactly

Raw predictors: load_id, pickup, delivery, pickup/delivery latitude and longitude, distance, equipment, weight, date, market_index, and quote_signal. The labeled file additionally contains posted_rate.

Final output verification: validation_predictions.csv contains 12,000 rows, zero missing values, zero duplicate IDs, zero non-positive predicted rates, and exact load_id order matching the provided prediction template.

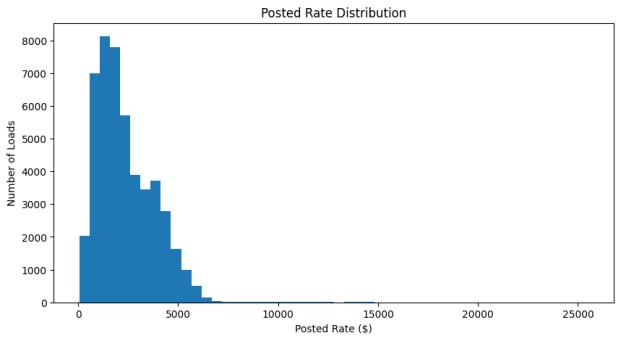
Why chronology matters

The final validation period occurs after all labeled data. A random split would mix past and future observations. Local evaluation therefore follows the real prediction direction: train on earlier months and validate on later months.

03 EDA - Target Distribution

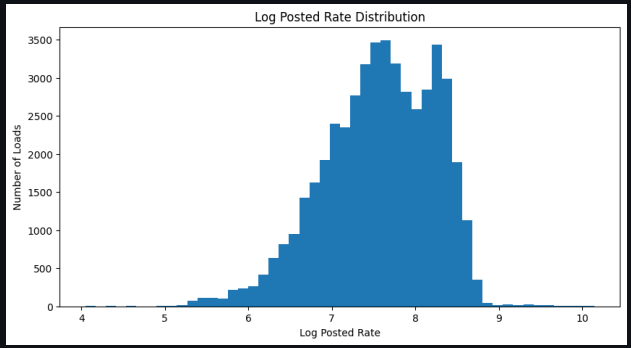
Understand scale and skew before choosing the modeling target.

EXACT NOTEBOOK OUTPUT



Code used: plt.hist(train['posted_rate'], bins=50). Raw posted_rate has a long right tail.

EXACT NOTEBOOK OUTPUT



Code used: plt.hist(np.log(train['posted_rate']), bins=50). The log transform compresses that tail.

Target statistic	Value
Mean	$2,373.98
Median	$2,030.76
25th percentile	$1,251.55
75th percentile	$3,330.75
Minimum	$57.22
Maximum	$25,533.00

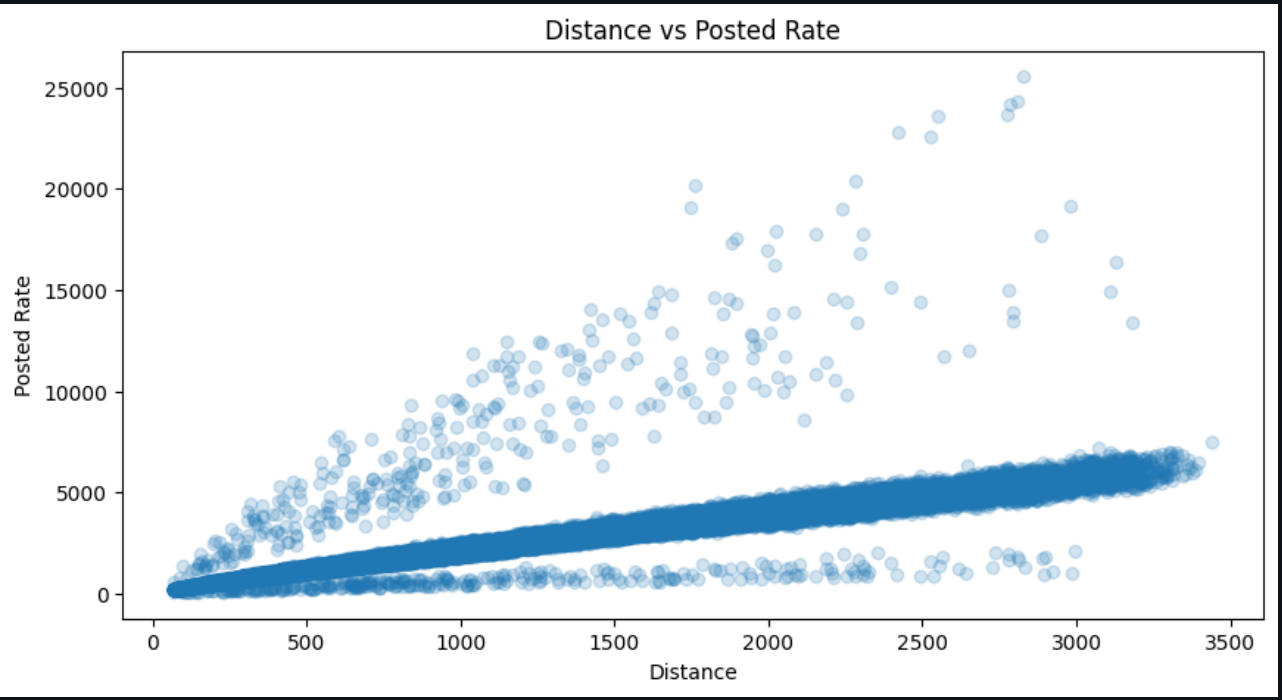
Modeling decision

The final local model is trained on log(posted_rate), and model outputs are transformed back to dollars with exp(). This is exactly how feature_engineering.ipynb and model.ipynb implement the target.

04 EDA - Distance & Freight Economics

Distance is the strongest simple linear driver of total posted rate.

EXACT NOTEBOOK OUTPUT



Exact notebook scatter: distance versus posted_rate.

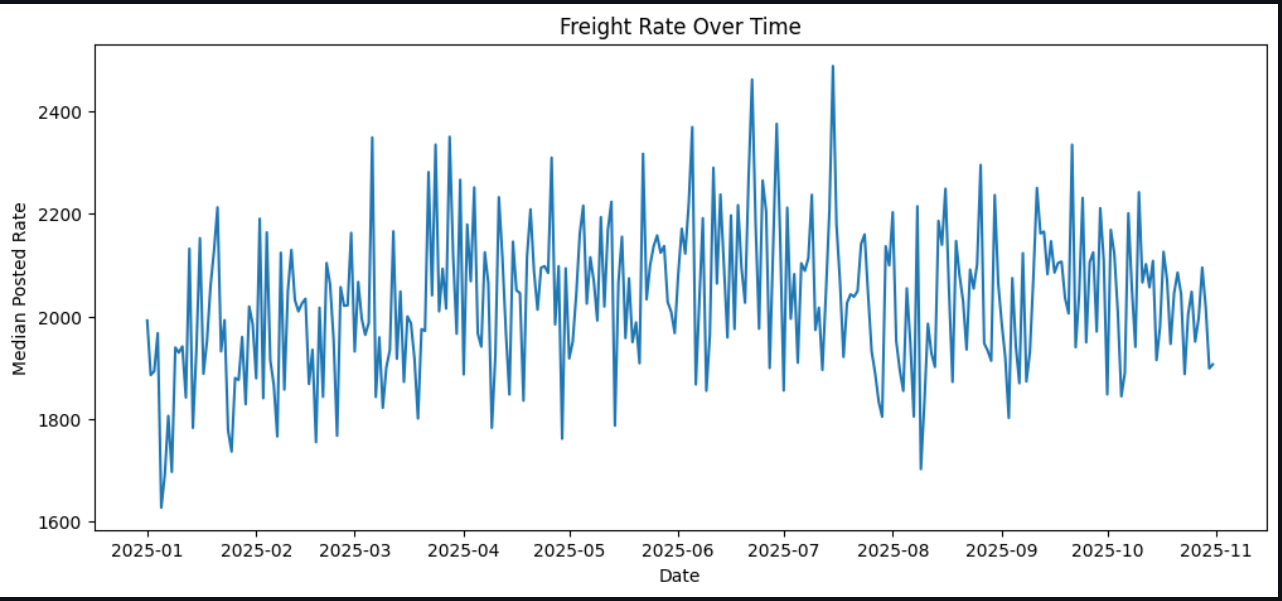
Variable	Correlation with posted_rate	Interpretation
distance	0.9085	Very strong positive linear relationship
weight	0.0348	Weak linear relationship by itself
market_index	0.0342	Weak linear relationship by itself
quote_signal	-0.0399	Weak negative linear relationship by itself

Low linear correlation was not treated as proof that a feature is useless. LightGBM can learn nonlinear thresholds and interactions, so market or quote signals may still matter jointly with lane, equipment, distance, or date.

05 EDA - Time & Future-Period Shift

Raw training and final-validation distributions are similar in some dimensions but not identical.

EXACT NOTEBOOK OUTPUT



Exact notebook line plot: daily median posted rate across the labeled training period.

Raw-input measure	Training	Validation
Mean distance	1,135.86	1,141.77
Median distance	953.30	953.60
Mean weight	31,028.84	30,506.64
Median weight	31,436.50	31,193.00
Mean market_index	1.0834	0.9269
Median market_index	1.0558	0.9229
Mean quote_signal	2.0625	2.0513

Observed shift

The raw market_index mean moves from 1.0834 in training to 0.9269 in validation. This supports chronological evaluation instead of random splitting.

06 Data Quality Audit

Raw issues, exact counts, and the treatment applied by feature engineering.

Issue	Training	Validation	Treatment
Missing weight	300	165	Create weight_missing; abs negative values; fill with training median
Negative weight	292	145	Convert to absolute value
Missing market_index	374	249	Create missing flag; fill with training median
Missing quote_signal	0	0	No imputation required
Duplicate training rows	0	-	No action required
Duplicate training load_id	0	-	No action required
Missing/non-positive posted_rate	0 / 0	-	All labeled targets usable

Imputation values used by feature_engineering.ipynb: after absolute-value correction, weight median = **31,496 lb**; raw training market_index median = **1.0558**.

Why missing flags are kept

Median filling avoids dropping loads. The accompanying missing indicators preserve whether the original value was absent instead of making an imputed value indistinguishable from a genuine observation.

07 Equipment, Geography & Lane Coverage

Equipment and route identity materially structure freight rates.

Equipment	Training rows	Mean posted rate	Median posted rate
Dry Van	27,202	$2,271.55	$1,953.04
Reefer	12,045	$2,553.64	$2,196.67
Flatbed	8,753	$2,445.09	$2,076.81

64
unique pickup locations

64
unique delivery locations

4,014
unique training lanes

4,214
unique validation lanes

Future lane novelty

10,539 of 12,000 validation rows (87.83%) use a lane seen in the full January-October labeled data, while 1,461 rows use a new lane. feature_engineering.ipynb creates a new_lane indicator and maps unseen validation lanes to UNKNOWN_LANE before saving processed CSVs.

08 Feature Engineering

What was created, why it was created, and what the final model actually consumes.

Group	Features	Purpose
Freight	distance, log_distance, weight, log_weight, weight_missing, equipment	Load scale and nonlinear size effects
Geography	pickup, delivery, pickup_lat, pickup_lon, delivery_lat, delivery_lon, lane	Origin/destination and route structure
Lane novelty	new_lane	Marks validation lanes unseen in full labeled history
Market	market_index, market_index_missing, quote_signal	Market conditions and quoting signal
Calendar	month, week_of_year, day_of_month, day_of_week, is_weekend	Seasonality and day effects

Verified feature-count detail

feature_engineering.ipynb creates and saves new_lane, but its local FEATURES list contains 21 columns and its final feature check therefore reports 21. model.ipynb explicitly adds new_lane to its FEATURES list; the LightGBM model therefore trains on 22 features.

Target leakage prevention: posted_rate is not a model input. rate_per_mile is used only for analysis/diagnostics because it directly contains posted_rate. load_id and the raw date column are also excluded from the model feature matrix.

09 Categorical Handling - Exact Implementation

What happens before saving CSVs and after model.ipynb reloads them.

feature_engineering.ipynb builds pickup/delivery category vocabularies from the union of train and unlabeled validation labels. It maps validation lanes not observed in the full labeled training set to UNKNOWN_LANE. No validation target is used because none is available.

CSV storage does not preserve pandas category vocabularies. model.ipynb reloads the processed CSVs, converts only the labeled training categorical columns to category dtype, fits the model, and then aligns final-validation categories to the training categories. As a result, previously unseen category values become categorical missing values during final prediction.

After final category alignment	Rows represented as categorical missing values
pickup	725
delivery	722
lane	1,461

Why final prediction still succeeds

LightGBM handles missing categorical values, while numeric distance, weight, coordinates, market signals, and calendar features remain available. The produced file has 12,000 complete positive predictions and passed score.py. For a stricter future pipeline, explicit UNKNOWN_PICKUP / UNKNOWN_DELIVERY / UNKNOWN_LANE tokens should be fitted consistently inside each historical split.

10 Time-Based Local Validation & Baseline

A chronological validation holdout is used for model selection and early stopping.

Partition	Rows	Dates	Role
Local model-fit period	38,477	2025-01-01 to 2025-08-31	Fit candidate model
Time-based validation holdout	9,523	2025-09-01 to 2025-10-31	Evaluate candidate and drive early stopping
Final unlabeled validation	12,000	2025-11-01 to 2025-12-31	Generate submission predictions only

The baseline predicts one constant value for every September-October validation row: the January-August median posted_rate of **$2,026.04**.

Baseline metric	Result
MAE	$1,148.92
RMSE	$1,569.42
MAPE	70.15%
WAPE	48.02%

Important terminology

September-October is a time-based validation holdout, not an untouched final test set, because it is also used for LightGBM early stopping and model selection. Spotter's hidden evaluation remains the truly unseen final assessment.

11 LightGBM Model

Model choice, exact training configuration, and early stopping behavior.

LightGBM is a gradient-boosted decision-tree model suited to structured tabular data. It can learn nonlinear thresholds and interactions across numeric and categorical inputs without requiring a neural network.

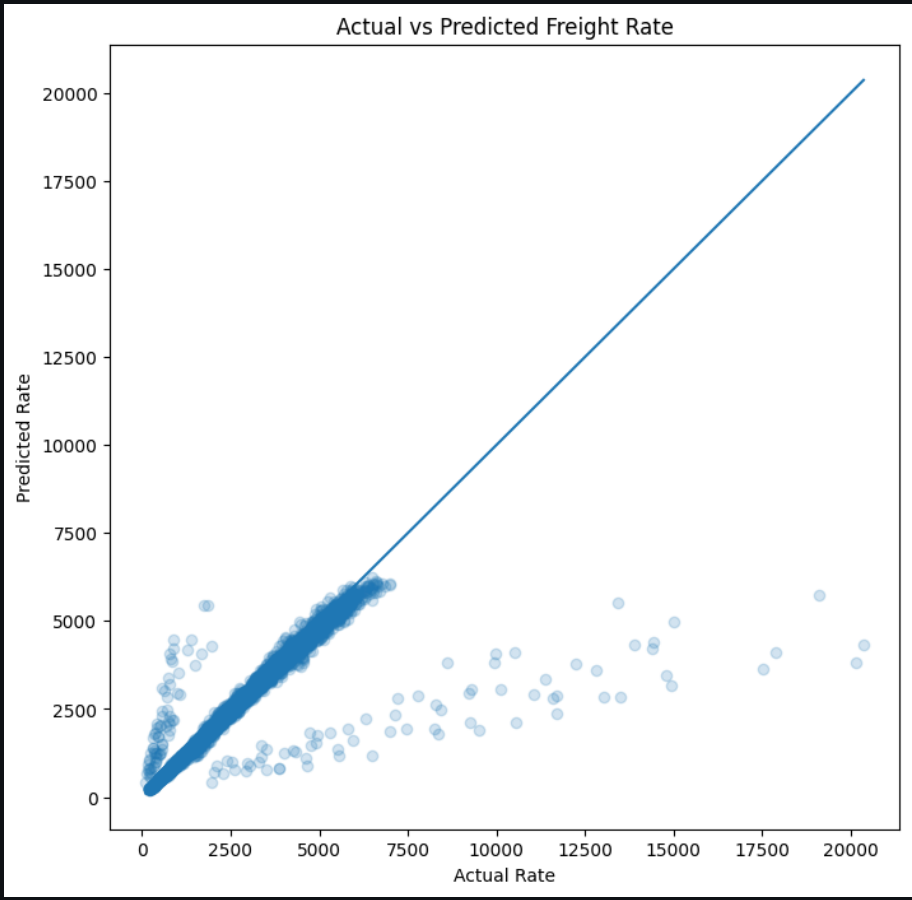
Parameter	Value
objective	regression
n_estimators	3000 maximum during local validation
learning_rate	0.03
num_leaves	31
max_depth	-1
min_child_samples	40
subsample / frequency	0.8 / 1
colsample_bytree	0.8
random_state	42
early stopping	100 rounds
best iteration	171

The candidate model trains on **log_posted_rate**. Early stopping monitors the September-October time-based validation holdout and selects iteration 171. The final model then retrains on all 48,000 labeled rows using 171 trees.

12 Local Validation Results

Exact evaluation output from model.ipynb - not Spotter hidden metrics.

EXACT NOTEBOOK OUTPUT



Exact notebook scatter. The diagonal indicates perfect predictions. Most ordinary-rate rows cluster near it, while rare extreme rates create much larger misses.

Model	MAE	RMSE	MAPE	WAPE
Median baseline	$1,148.92	$1,569.42	70.15%	48.02%
LightGBM	$129.79	$640.94	5.53%	5.42%

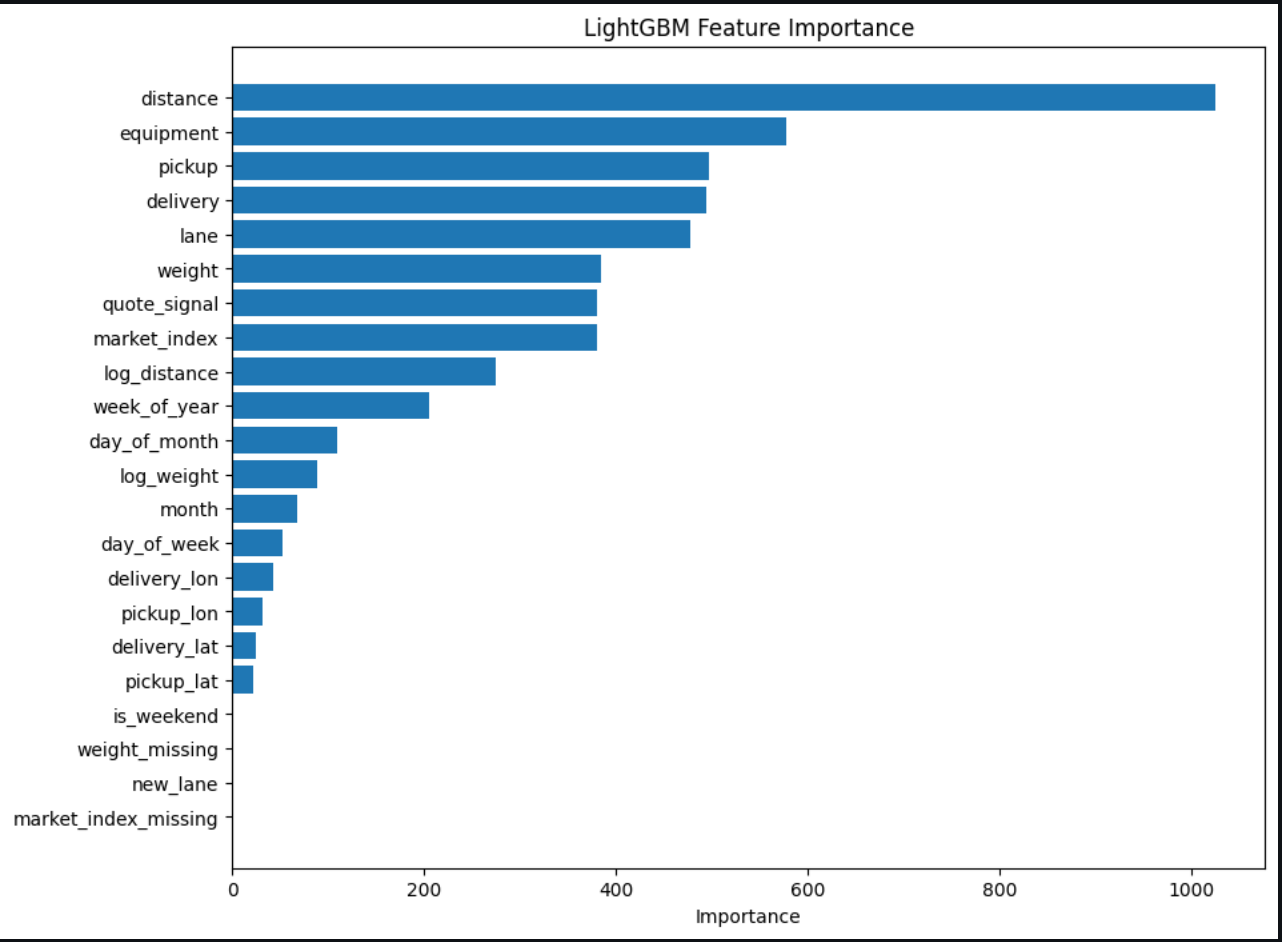
88.7% lower local-validation MAE than baseline

The substantially larger RMSE than MAE indicates that a small number of unusually large errors remain even though typical predictions are much closer.

13 Feature Importance

Exact split-count importance produced by the local LightGBM candidate model.

EXACT NOTEBOOK OUTPUT



Exact notebook horizontal bar chart. model.feature_importances_ is LightGBM split-count importance; it is not causal importance and is not the same as gain or SHAP importance.

Distance is the dominant split feature (1,025), followed by equipment (577), pickup (497), delivery (494), lane (477), weight (384), quote_signal (380), and market_index (380). This is consistent with freight pricing: trip length sets much of the price scale while equipment, route identity, load characteristics, and market conditions adjust it.

14 Error Analysis

Distribution and segmentation of local-validation error.

Absolute percentage-error percentile	Result
50th	2.49%
75th	4.29%
90th	6.66%
95th	8.81%
99th	75.22%
Maximum	477.47%

Equipment	Mean absolute error
Dry Van	$121.34
Flatbed	$131.16
Reefer	$147.68

Interpretation

Median percentage error is 2.49%, but the 99th percentile rises to 75.22%. A small number of extreme or unusual loads therefore dominate squared-error metrics and explain why RMSE remains much higher than MAE.

15 Final Retraining & Submission Predictions

What was generated after local model selection.

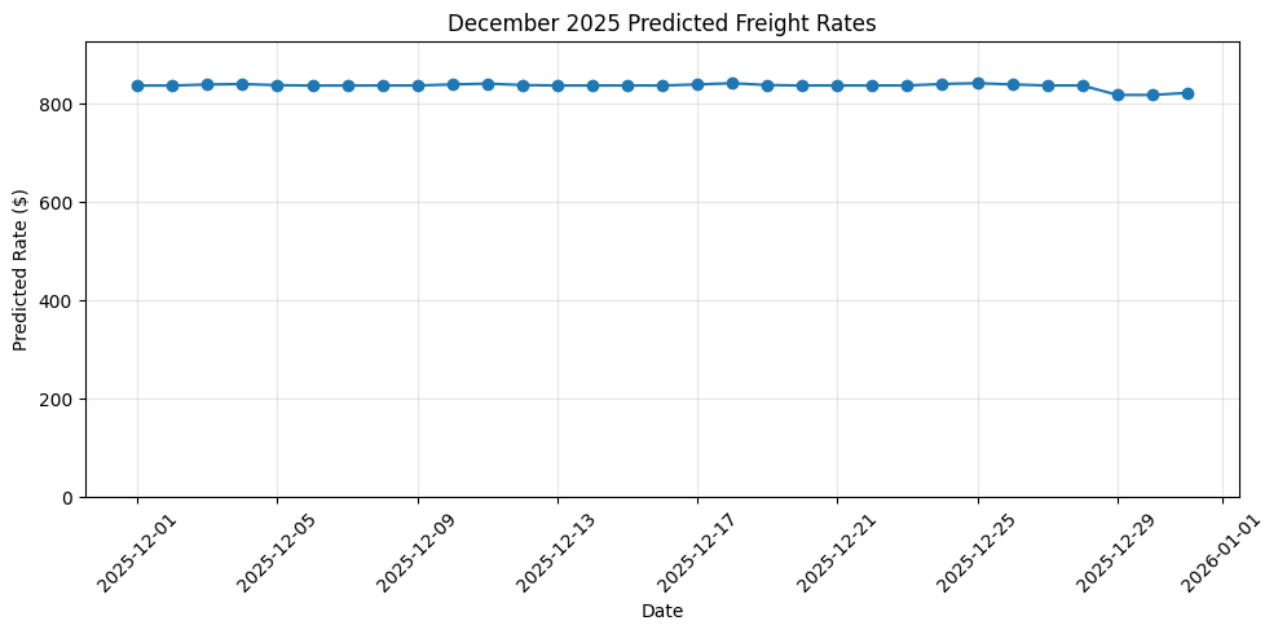
Final prediction check	Verified result
Rows	12,000
Columns	load_id, predicted_rate
IDs match template in exact order	True
Missing values	0
Duplicate load IDs	0
Non-positive rates	0
Predicted-rate mean	$2,338.99
Predicted-rate median	$2,032.48
Predicted-rate range	$214.99 to $6,295.77

After selecting 171 boosting iterations locally, the final model fits all 48,000 labeled rows. It predicts all 12,000 final-validation rows in log space and converts predictions back to dollars with np.exp(). The resulting file is outputs/validation_predictions.csv.

16 Fixed December Scenario

Exact notebook output plus the actual internal features used for the 31-day scenario.

EXACT NOTEBOOK OUTPUT



Exact model.ipynb plot: December 1-31, 2025 predictions for the fixed Lexington -> Fort Wayne shipment scenario.

Visible fixed input	Value
Pickup	Lexington
Delivery	Fort Wayne
Distance	360 miles
Equipment	Dry Van
Weight	32,000 lb
Dates	2025-12-01 through 2025-12-31
Rows	31
Predicted-rate range	$817.91 to $841.74

Precise implementation detail

In the scorer input schema, the shipment characteristics are fixed and the date is the only supplied field that changes. Internally, model.ipynb also derives calendar features from each date and joins that date's average market_index and quote_signal from the unlabeled December validation rows before predicting. No December target values are used.

17 Scorer Validation

The supplied local checker validates file schema/content and generates the candidate December chart.

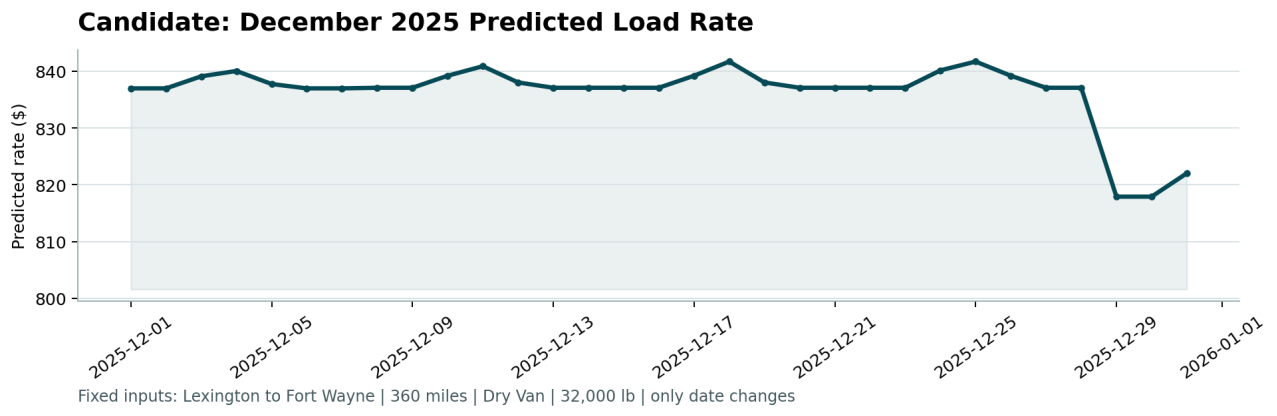
The December prediction file matches december_chart_inputs.csv on pickup, delivery, distance, equipment, weight, and date: **True**. The scorer was then run from the Windows project root.

```
py score.py --predictions outputs/validation_predictions.csv --december-predictions outputs/december_chart_predictions.csv
```

Terminal result

Validated 12,000 final predictions.
Validated 31 fixed December predictions.
Created chart: scorer_results\candidate_december.png
Final validation metrics are calculated by Spotter after submission.

EXACT SCORER OUTPUT



candidate_december.png created by score.py after both prediction files passed validation.

18 How to Run the Code

Tested Windows workflow that rebuilds outputs and reruns the supplied scorer.

The project was executed end-to-end from the root directory on Windows with Python 3.13.5. run.bat reran feature engineering, reran model.ipynb, regenerated the prediction files, and finally invoked score.py.

Packages used: pandas, numpy, matplotlib, scikit-learn, lightgbm, jupyter, nbconvert, and ipykernel.

```
1) Open Command Prompt / VS Code terminal in the project root
2) py -m pip install -r requirements.txt
3) run.bat

Manual scorer check (Windows, one line):
py score.py --predictions outputs/validation_predictions.csv --december-predictions outputs/december_chart_predictions.csv
```

Automated step	Purpose
feature_engineering.ipynb	Clean raw data, create engineered columns, save processed CSVs
model.ipynb	Chronological local validation, LightGBM training, final retraining, create both required prediction files
score.py	Validate 12,000 final predictions and 31 December rows, then create scorer chart

Reproducibility result

The tested run finished with SUCCESS after both scorer validations. The Windows ZMQ RuntimeWarning emitted during nbconvert did not stop execution and did not affect the generated outputs.

19 Step-by-Step Work Summary

What was done, why it was done, and the verified result at each stage.

Step	What we did	Why	Result
1. Load & inspect	Read train/validation; checked shapes, columns, dates, missingness, duplicates	Understand raw problem before modeling	48k labeled + 12k future rows
2. EDA	Target distribution, distance, time trend, market/quote, equipment, lanes	Find dominant patterns, skew, and future shift	Distance dominant; target skew; market shift
3. Clean	Abs negative weight; median fills; missing indicators	Retain rows while preserving missingness information	Processed numeric features complete
4. Engineer	Logs, lane, calendar, geography, market, novelty	Give the model useful nonlinear and domain structure	21 in FE check; 22 in model
5. Validate	Jan-Aug model fit; Sep-Oct time validation; median baseline	Simulate future prediction and select model	9,523-row validation holdout
6. Model	LightGBM on log target with early stopping	Strong tabular nonlinear model	Best iteration 171
7. Diagnose	Metrics, actual-vs-predicted, feature importance, equipment error	Understand strengths and failure modes	MAPE 5.53%; rare extreme misses
8. Retrain	Fit all 48k labeled rows with 171 trees	Use all labeled history for final submission model	Final candidate model
9. Predict	Create validation_predictions.csv + December predictions	Meet required output schema	12,000 + 31 rows
10. Score	Run supplied score.py	Verify file structure and fixed scenario	Both files validated

Verified from uploaded notebooks, CSV outputs, and scorer artifacts

20 Methodology Notes, Limitations & Improvements

Transparent distinctions between the submitted implementation and stricter future practice.

Area	Current implementation	Stricter future improvement
Preprocessing vs local holdout	Weight/market medians are calculated in feature engineering before the Jan-Aug / Sep-Oct model split	For strict local evaluation, fit all imputation statistics on Jan-Aug only, then apply them to Sep-Oct
new_lane in local validation	new_lane is defined relative to the full Jan-Oct labeled data, so every labeled row has new_lane=0	Recompute novelty relative to each historical training window
Category novelty	Unseen final categories become LightGBM categorical missing values after CSV reload/alignment	Use explicit UNKNOWN tokens and fit category vocabularies on each historical training fold
Model-selection holdout	Sep-Oct is used for both performance reporting and early stopping	Use rolling-origin tuning folds and preserve an untouched labeled month for final local testing
Extreme target rows	Rare loads create large squared errors	Compare robust/direct-dollar objectives using time-aware folds without deleting legitimate difficult rows
Importance	Split-count feature_importances_	Add gain importance or SHAP for deeper interpretation

Why the submission pipeline was frozen

The candidate already showed strong local validation performance, both required prediction files passed the supplied scorer, and run.bat reproduced the workflow successfully. Further changes should be evaluated with a stricter validation protocol rather than repeatedly optimized against the same September-October holdout.

21 Conclusion

A complete, reproducible, and transparently documented machine-learning assessment.

The project demonstrates an end-to-end freight-rate ML workflow: raw-data inspection, EDA, cleaning, domain-informed feature engineering, time-aware local validation, baseline comparison, LightGBM modeling, error analysis, final retraining, fixed-scenario prediction, scorer validation, and one-command reproducibility.

48,000	12,000	31	88.7%
rows used for final training	final predictions validated	December rows validated	local MAE reduction vs baseline

Final status

Local ML pipeline: COMPLETE. Reproducibility test: PASSED. validation_predictions.csv: PASSED score.py. december_chart_predictions.csv: PASSED score.py. Hidden Spotter evaluation: pending external submission.

Technical stack: Python | pandas | NumPy | Matplotlib | scikit-learn metrics | LightGBM | Jupyter | nbconvert | Windows batch automation | supplied score.py